Lesson 14 Teacher's Key: Raising the Stakes

001 Game Creator (Windows only) - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

This engine's version of difficulty scaling looks different from the other seven: since enemies are always pre-placed at design time (never runtime-spawned, per Lesson 5's precedent), 'more enemies per wave' means activating MORE pre-placed copies, not creating new ones - an honest, confirmed-mechanism-only approach given this engine's unverified runtime speed API.

Level 1 - Full Worked Solution

- enemy_6 and enemy_7 are added at design time, in the Map editor, as two more copies alongside the original enemy_1..enemy_5, spaced with the same full-enemy-width gap as the rest of the row - a direct continuation of Lesson 5's design-time-placement precedent.
- SpawnWave's nested If(wave >= 2) / If(wave >= 3) branches are what actually implements the scaling - each wave activates one more already-existing copy than the last, using only the confirmed Revive/Teleport mechanism from Lesson 11.
- ResetGame Destroying enemy_6/7 by default (before SpawnWave runs) is what keeps wave 1 looking exactly like Lesson 13's game - only 5 active copies until later waves need more.

Why This Structure

- Rather than fabricate an unconfirmed 'set movement speed at runtime' action, this lesson leans entirely on mechanisms already verified in this course (Spawn/Destroy/Revive, design-time Actor placement) - the same honest-uncertainty approach used for this engine's restart flow in Lesson 10 and hit-flash timing in Lesson 13.
- This actually turns this engine's fixed-actor-count constraint into a genuine, different-but-valid solution to the same problem every other engine solves at runtime - worth presenting to students as a real design trade-off, not a lesser version of the 'real' approach.

Level 2 / Level 3 Talking Points

- Ask students on this engine to explain, in their own words, why 001 Game Creator's approach to more-enemies-per-wave is structurally different from every other engine this week, and why that's a reasonable consequence of a design decision made all the way back in Lesson 5.

- For advanced students: if their installed version DOES expose a runtime movement-speed action (check together), implementing per-wave speed scaling as a Level 3 stretch is a great extension, using the exact same wave-gated If-branch pattern as the count scaling.

Common Student Mistakes

- Forgetting to Destroy enemy_6/7 in ResetGame, leaving them visible on a fresh wave 1 game.
- Using If(wave = 2) instead of If(wave >= 2) for enemy_6, which would deactivate it again once wave 3 begins instead of keeping it active.
- Naming the new copies inconsistently (not exactly enemy_6/enemy_7), breaking SpawnWave's references to them.